

Assincronicidade e Modularização em JavaScript

Linguagem de Programação III

Definição de Código Síncrono

- Código síncrono executa instruções uma após a outra.
- Cada linha espera a anterior terminar.
- Modelo simples, porém bloqueante.

Exemplo de Código Síncrono

```
console.log('Início');  
console.log('Processando');  
console.log('Fim');
```

Problema do Modelo Síncrono

- Se uma operação demorar, o programa trava.
- Exemplo: consulta ao banco de dados.
- Backend moderno precisa evitar bloqueio.

Definição de Código Assíncrono

- Permite executar tarefas demoradas sem bloquear o programa.
- Muito utilizado em servidores e APIs.
- Baseado em eventos e Promises.

Antes vamos conectar os
conceitos...

Organização do Código em Backend

- Aplicações backend geralmente dividem o código em camadas.
- Cada camada possui uma responsabilidade específica.
- Isso melhora organização, manutenção e reutilização do código.

Arquitetura em Camadas

- Controller
- Service
- Repository
- Cada camada executa uma função específica dentro do sistema.

Controller → Service → Repository

- Fluxo típico de uma aplicação backend:
- Cliente → Controller → Service → Repository → Banco de Dados
- Controller recebe a requisição.
- Service aplica **regras de negócio**.
- Repository acessa os dados.

Exemplo Simplificado de Fluxo

- 1 O cliente solicita dados.
- 2 O Controller recebe a requisição.
- 3 O Service processa a lógica.
- 4 O Repository consulta os dados.
- 5 A resposta retorna ao cliente.

O que é uma API?

- API significa Application Programming Interface.
- Permite que sistemas diferentes se comuniquem.
- Frontends e aplicativos usam APIs para acessar dados.

Exemplo de Resposta de API (JSON)

```
{  
  "id": 1,  
  "nome": "Ana"  
}
```

Promise

O que é uma Promise?

- Objeto que representa um valor futuro.
- Possui três estados:
- Pending (pendente)
- Fulfilled (resolvida)
- Rejected (rejeitada).

Estrutura de uma Promise

```
const promessa = new Promise((resolve, reject) => {  
  const sucesso = true;  
  if (sucesso) {  
    resolve('Operação concluída');  
  } else {  
    reject('Erro na operação');  
  }  
});
```

Consumindo Promise com then/catch

- `then()` executa quando resolve.
- `catch()` executa quando ocorre erro.
- Pode gerar código difícil de ler.

Exemplo com then/catch

promessa

```
.then(resultado => console.log(resultado))  
.catch(erro => console.log(erro));
```

Problema do Encadeamento

- Múltiplas Promises geram código confuso.
- Difícil manutenção.
- Solução moderna: `async/await`.

async & await

Definição Formal de async

- Palavra-chave usada antes de uma função.
- Toda função async retorna uma Promise.
- Permite uso de await dentro dela.

Definição Formal de await

- Só pode ser usado dentro de função async.
- Faz a função esperar a Promise resolver.
- Não bloqueia o restante do programa.

Exemplo Básico async/await

```
function buscarDados() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve('Dados recebidos');  
    }, 2000);  
  });  
}  
  
async function executar() {  
  const resultado = await buscarDados();  
  console.log(resultado);  
}  
  
executar();
```

Por que async/await é Melhor?

- Código mais legível.
- Fluxo semelhante ao código síncrono.
- Facilita manutenção e depuração.

try/catch

try/catch com async/await

- Quando usamos await, precisamos tratar erros.
- Se Promise for rejeitada, gera exceção.

Tipos de Erros em JavaScript

- `SyntaxError` → erro de sintaxe.
 - `ReferenceError` → variável não definida.
 - `TypeError` → tipo incorreto.
 - Error personalizado.

Exemplo de Erro Sem Tratamento

```
function dividir(a, b) {  
    return a / b;  
}
```

```
console.log(dividir(10, 0));
```

Problema de Não Tratar Erros

- Aplicação pode parar inesperadamente.
- Usuário recebe mensagem técnica.
- Sistema fica instável.

Definição de try/catch

- try → bloco onde colocamos código que pode gerar erro.
- catch → captura o erro caso ocorra.
- Permite continuar execução do programa.

Exemplo Prático Simples

```
try {  
    let resultado = 10 / 0;  
    console.log(resultado);  
} catch (erro) {  
    console.log('Erro detectado');  
}
```

Objeto de Erro

- Erro possui propriedades:
- name → tipo do erro.
- message → mensagem do erro.
- stack → rastreamento da pilha.

Acessando Propriedades do Erro

```
try {  
    naoExiste();  
} catch (erro) {  
    console.log(erro.name);  
    console.log(erro.message);  
}
```


Throw – Lançando Erros

- throw permite criar erro manualmente.
- Muito usado em regras de negócio.

Exemplo com throw

```
function validarIdade(idade) {  
  if (idade < 18) {  
    throw new Error('Idade mínima não atingida');  
  }  
  return 'Acesso permitido';  
}
```

Exemplo Assíncrono Completo

```
function buscarUsuario() {  
  return new Promise((resolve, reject) => {  
    reject('Usuário não encontrado');  
  });  
}
```

```
async function executar() {  
  try {  
    const usuario = await buscarUsuario();  
    console.log(usuario);  
  } catch (erro) {  
    console.log('Erro capturado:', erro);  
  }  
}
```

```
executar();
```

Boas Práticas em Backend

- Sempre usar try/catch em operações assíncronas.
- Não expor erro técnico ao usuário final.
- Registrar erro para análise futura.
- Separar erro técnico de erro de negócio.

Tratamento de Erros com try/catch

- Backend precisa capturar erros.
- Evita que aplicação pare.
- Permite retornar resposta adequada.

Exemplo Completo com try/catch

```
async function executar() {  
  try {  
    const resultado = await buscarDados();  
    console.log(resultado);  
  } catch (erro) {  
    console.log('Erro capturado:', erro);  
  }  
}
```

Regras Importantes

- `await` só funciona dentro de função `async`.
- `async` sempre retorna `Promise`.
- Sempre tratar erros com `try/catch`.

Aplicação em Backend

- Consultas ao banco são assíncronas.
- Requisições HTTP são assíncronas.
- Leitura de arquivos é assíncrona.

Modularização

O que é Modularização?

- Dividir o código em partes menores (módulos).
- Cada módulo possui uma responsabilidade.
- Evita arquivos grandes e desorganizados.
- Facilita manutenção e reutilização.

Por que modularizar?

- Organização do projeto.
- Separação de responsabilidades.
- Facilidade para testar partes isoladas.
- Base da arquitetura MVC.

O que é um Módulo em JavaScript?

- Um arquivo JavaScript é considerado um módulo.
- Ele pode exportar funções, classes ou variáveis.
- Outros arquivos podem importar esse conteúdo.

Import & Export

Definição de export

- export torna algo público.
- Permite que outros arquivos utilizem esse código.
- Pode ser nomeado ou default.

Regras do Export Nomeado

- Deve usar { } ao importar.
- Pode exportar várias funções no mesmo arquivo.
- Nome deve ser idêntico ao exportado.

Export Nomeado – Exemplo

```
// arquivo: calculadora.js
```

```
export function somar(a, b) {  
  return a + b;  
}
```

```
export function subtrair(a, b) {  
  return a - b;  
}
```


Importando Export Nomeado

```
// arquivo: app.js
```

```
import { somar, subtrair } from './calculadora.js';
```

```
console.log(somar(2, 3));
```

Definição de export default

- Permite exportar apenas um elemento principal.
- Não exige { } na importação.
- Muito usado para classes e serviços.

Export Default – Exemplo

```
// arquivo: usuarioService.js
```

```
export default class UsuarioService {  
  buscar() {  
    return 'Usuário encontrado';  
  }  
}
```

Importando Export Default

```
// arquivo: app.js
```

```
import UsuarioService from './usuarioService.js';
```

```
const service = new UsuarioService();
```

```
console.log(service.buscar());
```

Diferença Entre Export Nomeado e Default

- Nomeado → pode exportar vários itens.
- Default → apenas um por arquivo.
- Nomeado exige { }, default não exige.

Erro Comum de Iniciantes

- Esquecer a extensão .js na importação.
- Confundir export nomeado com default.
- Caminho incorreto do arquivo.

Estrutura Real de Backend

- userRepository.js → acesso a dados.
- userService.js → regras de negócio.
- userController.js → recebe requisições.
- app.js → inicialização.

Exemplo Arquitetural Simples

```
// userRepository.js
```

```
export function buscarUsuario(id) {  
  return { id: id, nome: 'Ana' };  
}
```


Service Importando Repository

```
// userService.js
```

```
import { buscarUsuario } from './userRepository.js';
```

```
export function obterUsuario(id) {  
  return buscarUsuario(id);  
}
```

App Importando Service

```
// app.js
```

```
import { obterUsuario } from './userService.js';
```

```
console.log(obterUsuario(1));
```

Conectando os Conceitos da Aula

- Nesta aula relembremos/aprendemos vários conceitos importantes:
 - Promise
 - async / await
 - try / catch
 - import / export
 - modularização

Agora vamos usar esses conceitos para simular um backend na Lista de Exercícios 1